

High Performance Communications

Lecture #3

Gil Bloch

Lectures Outline

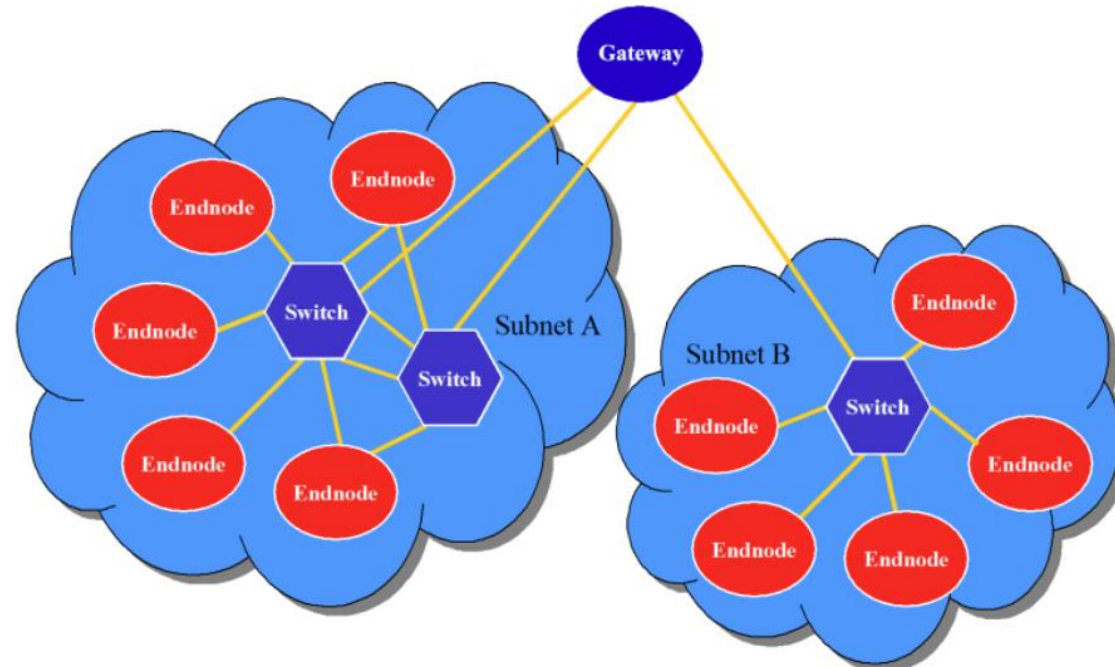
1. Network Metrics
2. Socket Programming
3. User-level Networking
4. Verbs Essentials

Part 4:

Verbs Essentials

Before We Start – InfiniBand

- SDN Network Architecture
- Designed for Datacenters / Local Area Networks
- Designed for low-latency, high bandwidth



What Is a Subnet Manager (SM)?

- The Subnet Manager (SM) is a software entity that configures its local subnet and ensures its continued operation. It sets up primary and secondary paths between every end point so that traffic flow forwarding decisions are preprogrammed and data arrives in the least amount of time.
- There must be at least one SM present in a subnet in order to manage all switch and router setups, and to reconfigure the subnet when a link goes down or a new link comes up. The SM can reside on any of the devices within the subnet.
- All devices in the subnet must contain a dedicated Subnet Management Agent (SMA), which is used by the SM to communicate with the various InfiniBand components. There can be multiple SMs in a subnet, as long as only one is active at any moment.

What Are LIDs, GIDs, and GUIDs?

- All devices in a subnet have a Local Identifier (LID), a 16-bit address assigned by the Subnet Manager.
- All packets sent within a subnet use the LID as the destination address for forwarding and switching packets at the Link Level. The LIDs allow for up to 48,000 end nodes within a single subnet.
- When a subnet is reconfigured, new LIDs are assigned to the various endpoints within the subnet. Routing between different subnets is done on the basis of a Global Identifier (GID), a 128-bit address modeled after IPv6 addresses, which allows for InfiniBand's essentially unlimited scalability. GIDs identify an end node, port, switch, or multicast group.
- Global Unique Identifiers (GUID) are 64-bit definitions for all the elements within a subnet, including chassis, HCAs, switches, routers, and ports. The GUID never changes, and is used as part of the address for creating a GID. GIDs and GUIDs are independent of LIDs and are therefore immune to subnet reconfiguration.

The “Verbs” Library

- Verbs is an abstract description of the functionality that is provided for applications for using RDMA.
 - Verbs is not an API
 - There are several implementations for it
- Verbs can be divided into two major groups
 - Control path – manage the resources and usually requires context switch
 - Create
 - Destroy
 - Modify
 - Query
 - Work with events
 - Data path – Use the resources to send/receive data and doesn't require context switch
 - Post Send
 - Post Receive
 - Poll CQ
 - Request for completion event

Why use verbs?

- Verbs is a low level description for RDMA programming
 - Verbs are close to the “bare-metal” and provide best performance
 - Latency
 - BW
 - Message rate
 - Verbs can be used as building blocks for many applications
 - Sockets
 - Storage
 - Parallel computing
- Any other level of abstraction over verbs may harm the performance
- Once you know it, verbs are not so mysterious ...

libibverbs

- libibverbs, developed and maintained by Roland Dreier since 2006, are de-facto the verbs API standard in *nix
 - Developed as an Open source
 - The kernel part of the verbs is integrated in the Linux kernel since 2005 – Kernel 2.6.11
 - Inbox in several *nix distributions
 - There are low-level libraries from several HW vendors
- Same API for all RDMA-enabled transport protocols
 - **InfiniBand** – Networking architecture which supports RDMA
 - requires both NICs and switches that supports it.
 - **RDMA Over Converged Ethernet (RoCE)** – encapsulation of RDMA packets over Ethernet/IP frames
 - requires NICs which supports it and standard Ethernet switches
 - A soft-RoCE implementation can be used with standard Ethernet NICs
 - **Internet Wide Area RDMA Protocol (iWARP)** – provides RDMA over Stream Control Transmission Protocol (SCTP) and Transmission Control Protocol (TCP)
 - requires NICs which supports it and standard Ethernet switches

General tips on libibverbs

- Source code that uses libibverbs should include the header:
 - `#include <infiniband/verbs.h>`
- Executables/libraries that work with libibverbs should be linked with:
 - `-libverbs`
- All input structures should be zeroed
 - Using `memset()` or structure initialization
 - If the structure will be extended in the future, the value zero will keep the legacy behavior
- Most resource handles are pointers, so using bad handles may cause segmentation fault
- Verbs that return a pointer – return a valid value in case of a success and NULL in case of a failure
- Verbs that return an integer – return zero in case of a success and -1 or `errno` in case of a failure
 - For more information, read the documentation of each verb

Memory Region (MR)

- Memory Region is a virtually contiguous memory block that was registered, i.e. prepared for work with RDMA.
 - Any memory buffer in the process' virtual space can be registered
 - Available permissions. One or more of the following permissions (Or'ed):
 - Local operations (Local Read is always supported)
 - IBV_ACCESS_LOCAL_WRITE
 - IBV_ACCESS_MW_BIND
 - Remote operations
 - IBV_ACCESS_REMOTE_WRITE
 - IBV_ACCESS_REMOTE_READ
 - IBV_ACCESS_REMOTE_ATOMIC
 - If Remote Write or Remote Atomic is enabled, local Write should be enabled too
 - The same memory buffer can be registered multiple times
 - even with different permissions
 - After a successful memory registration, two keys are being generated:
 - Local Key (lkey)
 - Remote Key (rkey)
- Those keys are used when referring to this MR in a Work Request

Queue Pair (QP)

- Queue Pair is the actual object that transfers data
 - It encapsulates both Send and Receive Queue
 - Each of them is completely independent
 - Send Queue can generate Work Completion for every Send Request or for specific Send Requests
 - Receive queue generates Work Completion for every completed Receive Request
 - Full duplex
 - A QP represent a real HW resource
- There are three major transport types
 - Reliable Connected (RC)
 - An RC QP is connected to a single RC QP
 - Reliability is guaranteed (ordering, integrity and arrival of all packets)
 - Supports operations that need ACK
 - Unreliable Connected (UC)
 - An UC QP connected to a single UC QP
 - Reliability is not guaranteed
 - Unreliable Datagram (UD)
 - An UD QP can send/receive messages to/from any UD QP
 - Reliability is not guaranteed
 - Multicast is supported
 - Each message is limited to one packet

Queue Pair (QP) (cont.)

Metric	UD	UC	RC
Reliability			😊
Send (with immediate)	😊	😊	😊
RDMA Write (with immediate)		😊	😊
RDMA Read			😊
Atomic operations			😊
Multicast	😊		
Max message size	MT U	2GB	2GB
CRC	😊	😊	😊

Memory Region (MR): API

- `struct ibv_mr *ibv_reg_mr(struct ibv_pd *pd,`

`void *addr, size_t length,`
`enum ibv_access_flags access);`

 - Register a memory buffer with specific permissions
 - Notice the following fields in `struct ibv_mr`:
 - `lkey` - The local key of this MR
 - `rkey` - The remote key of this MR
 - `addr` – The start address of the memory buffer that this MR registered
 - `length` – The size of the memory buffer that was registered
- `int ibv_dereg_mr(struct ibv_mr *mr);`
 - Deregister a Memory Region
 - This verb should be called if there is no outstanding Send Request or Receive Request that points to it

Connecting QPs

- Communication should be established between the connected QPs
 - Each side needs to know who is the other side
 - Each side needs to have information about the other side and the path to it
 - Each side needs to configure attributes that describe the send attributes
- Problem: How to connect QP X with QP Y?
 - We cannot transfer the needed information to establish the connection until the connection has already been established between them ...
- Solutions:
 1. Exchange information Out Of Band
 - For example: over sockets
 2. Use Communication Manager (CM) ← **this is the right way to connect QPs**

Connecting QPs (cont.)

- The following information needs to be exchanged when connecting QPs
 - QP number
 - LID number
 - RQ Packet Serial Number (PSN)
 - GID (if GRH is used)
- Path MTU must be equal on both sides
- If RDMA opcodes are used, the permissions of QP and MR should be configured to support them
- In each QP state transition, the relevant attributes to enable the state functionality needs to be configured
 - There are different attributes for every transport type
 - For RC QPs: retransmission count and timers
 - For RC/UC QPs: Primary path and alternate path (optional)

-
- LID - Local Identifier , GID – Group Identified, GRH - Global Route Headers,

Queue Pair (QP): API

- `struct ibv_qp *ibv_create_qp(struct ibv_pd *pd,`
 - `struct ibv_qp_init_attr *qp_init_attr);`
 - Create a new Queue Pair
 - Notice the following fields in `struct ibv_qp`:
 - `qp_num` – The physical QP number
 - `qp_context` – The private context that the QP is associated with
- `int ibv_destroy_qp(struct ibv_qp *qp);`
 - Destroy a Queue Pair
 - This verb should be called after detach is from all multicast groups
- `int ibv_modify_qp(struct ibv_qp *qp, struct ibv_qp_attr *attr,`
 - `enum ibv_qp_attr_mask attr_mask);`
 - Modify the QP attributes
- `int ibv_query_qp(struct ibv_qp *qp, struct ibv_qp_attr *attr,`
 - `enum ibv_qp_attr_mask attr_mask,`
 - `struct ibv_qp_init_attr *init_attr);`
 - Query the attributes of a QP

Queue Pair (QP): API (cont.)

```
struct ibv_qp_cap {
    uint32_t max_send_wr; //The number of Send Requests that can be outstanding in the QP
    uint32_t max_recv_wr; //The number of Receive Requests that can be outstanding in the QP
    uint32_t max_send_sge; //The number of S/G entries that each Send Request may hold
    uint32_t max_recv_sge; //The number of S/G entries that each Receive Request may hold
    uint32_t max_inline_data; //The requested inline data (in bytes) to be sent
};

struct ibv_qp_init_attr {
    void *qp_context; //A private context that the QP will be associated with
    struct ibv_cq *send_cq; //The CQ to be associated with the QP's Send Queue
    struct ibv_cq *recv_cq; //The CQ to be associated with the QP's Receive Queue
    struct ibv_srq *srq; //Optional: if not NULL, the SRQ to be associated with
    struct ibv_qp_cap cap; //The QP attributes to be created
    enum ibv_qp_type qp_type; //The QP transport type
    int sq_sig_all; //Set for every completed Send Request to generate a Work Completion
};
```

Queue Pair (QP): example

```
struct ibv_pd *pd;
struct ibv_cq *cq;
struct ibv_qp *qp;
struct ibv_qp_init_attr init_attr = {
    .send_cq = cq,
    .recv_cq = cq,
    .cap = {
        .max_send_wr = 1,
        .max_recv_wr = rx_depth,
        .max_send_sge = 1,
        .max_recv_sge = 1
    },
    .qp_type = IBV_QPT_RC
};

qp = ibv_create_qp(pd, &init_attr);
if (!qp) {
    fprintf(stderr, "Error, ibv_create_qp() failed\n");
    return -1;
}
```

Scatter/Gather (S/G) elements

- Every Work Request contains usually one or more S/G entries
 - Every S/G entry refers to a Memory Region or part of it
 - No S/G entries means zero-byte message
 - Gather – when local data is read and sent over the wire
 - Scatter – when data is received and written locally

```
struct ibv_sge {  
  
    uint64_t addr; // Start address of the memory buffer (usually  
    registered memory)  
  
    uint32_t length; // Size (in bytes) of the memory buffer  
  
    uint32_t lkey; // lkey of Memory Region that is associated  
    with this memory buffer  
  
};
```

Post Send Request

- Add a Send Request to the Send Queue
 - No context switch will occur
 - The HW will process it according to its scheduling algorithm
- Specify the attributes of the data transfer
 - How data will be sent (opcode, attributes)
 - How much data will be sent
 - Which local memory buffer(s) to read/write to
 - Depends on the opcode
 - If RDMA: the remote memory buffer attributes
 - If atomic: the remote memory buffer attributes and needed operands
 - If UD QP: information on how to reach to remote side
- Every Send Request is considered outstanding until a work Completion was generated for it or for other Send Request that followed it
 - While a Send Request is outstanding, the resources that this Send Request use must not be destroyed/(re)used
 - The content of memory buffers that their content will be filled is undefined
 - The memory buffers that their content is sent must be available
 - For UD QPs: Address Handles must be available

Post Send Request: API

- `int ibv_post_send(struct ibv_qp *qp, struct ibv_send_wr *wr,`

- `struct ibv_send_wr **bad_wr);`

- Add a linked list of Send Requests to the Send Queue

- Warning: `bad_wr` is mandatory; It will be assigned with the address of the Send Request that its posting failed

```
struct ibv_send_wr {
```

```
    uint64_t wr_id;                - Private context that will be available in the corresponding Work Completion
```

```
    struct ibv_send_wr *next;      - Address of the next Send Request. Should be NULL in the last Send Request
```

```
    struct ibv_sge *sg_list;       - Array of scatter/gather elements
```

```
    int num_sge;                  - Number of elements in sg_list
```

```
    enum ibv_wr_opcode opcode;     - The opcode to be used
```

```
    int send_flags;               - Send flags. Or of the following flags:
```

```
        IBV_SEND_FENCE – Prevent process this Send Request until the processing of previous RDMA  
                        Read and Atomic operations were completed.
```

```
        IBV_SEND_SIGNALED – Generate a Work Completion after processing of this Send Request ends
```

```
        IBV_SEND_SOLICITED – Generate Solicited event for this message in remote side
```

```
        IBV_SEND_INLINE - allow the low-level driver to read the gather buffers
```

```
    uint32_t imm_data;            - Send message with immediate data (for supported opcodes); extra 32 bits, in network  
                                order, that will be available in remote's Work Completion
```

Post Send Request: API (cont.)

```
union {
    struct {
        uint64_t remote_addr;
        uint32_t rkey;
    } rdma;
    struct {
        uint64_t remote_addr;
        uint64_t compare_add;
        uint64_t swap;
        uint32_t rkey;
    } atomic;
    struct {
        struct ibv_ah *ah;
        uint32_t remote_qpn;
        uint32_t remote_qkey;
    } ud;
} wr;
};
```

- Attributes for RDMA Read and write opcodes

- Remote start address (the message size is according to the S/G)

- rkey of Memory Region that is associated with remote memory

- Attributes for Atomic opcodes

- Remote start address (the message size is according to the S/G)

- Value to compare/add (depends on opcode)

- Value to swap if the comparison passed

- rkey of Memory Region that is associated with remote memory

- Attributes for UD QP

- Address Handle to get to remote side

- Remote QP number (of 0xffffffff for multicast message)

- Remote Q_Key value

Post Receive Request

- Add a Receive Request to the Receive Queue
 - No context switch will occur
 - The HW will process it according to its scheduling algorithm
- Specify where incoming message that needs Receive Request will be saved
 - The local memory buffer(s) to write to
 - Each incoming message will consume one Receive Request
 - The S/G list must be able to hold the incoming message
 - If the message was received on a UD QP
 - Extra 40 bytes should be added to the scatter list (for the Global Routing Header (GRH))
 - The message data will start at offset 40
- Every Receive Request is considered outstanding until a work Completion was generated to it
 - While a Receive Request is outstanding, the resources that this Receive Request use mustn't be destroyed/(re)used
 - The content of memory buffers that their content will be filled is undefined

Post Receive Request: API

- `int ibv_post_recv(struct ibv_qp *qp, struct ibv_recv_wr *wr, struct ibv_recv_wr **bad_wr);`
 - Add a linked list of Receive Requests to the Receive Queue
 - Warning: `bad_wr` is mandatory; It will be assigned with the address of the Receive Request that its posting failed

```
struct ibv_recv_wr {  
    uint64_t wr_id;           - Private context that will be available in the  
    corresponding Work Completion  
    struct ibv_recv_wr *next; - Address of the next Receive Request. Should be NULL  
    in the last Receive Request  
    struct ibv_sge *sg_list;   - Array of scatter elements  
    int num_sge;               - Number of elements in sg_list  
};
```

Polling for Work Completion

- Polling for Work Completion checks if the processing of a Work Request has ended
- A Work Completion holds information about a completed Work Request
 - Every Work Completion contains information about the corresponding completed Work Request
- Every Work Completion contain several attributes
 - The following fields are always valid (even if the Work Completion was ended with error)
 - *wr_id*
 - *status*
 - *qp_num*
 - *vendor_err*
 - The rest of the fields depend on the QP's transport type, opcode and status
- Work Completion of Send Requests:
 - Mark that a Send Request was performed and its memory buffers can be (re)used
 - For reliable transport QP: this means that the message was written in the buffers (if status is successful)
 - For unreliable transport QP: this means that the message was sent from the local port
- Work Completion of Receive Requests:
 - Mark that an incoming message was completed and its memory buffers can be (re)used
 - Contains some attributes about the incoming message, such as size, origin, etc.

Polling for Work Completion: API

- `int ibv_poll_cq(struct ibv_cq *cq, int num_entries, struct ibv_wc *wc);`
 - Read one or more Work Completions from a CQ and remove them from the CQ
- If the return value is non-negative – this is the number of polled Work Completions
- If the return value is negative – error occurred

```
struct ibv_wc {  
    uint64_t wr_id;                - Private context that was posted in the corresponding Work Request  
    enum ibv_wc_status status;     - The status of the Work Completion  
    enum ibv_wc_opcode opcode;    - The opcode of the Work Completion  
    uint32_t vendor_err;          - Vendor specific error syndrome  
    uint32_t byte_len;            - Number of bytes that were received  
    uint32_t imm_data;            - Immediate data, in network order, if the flags indicate that such exists  
    uint32_t qp_num;              - The local QP number that this Work Completion ended in  
    uint32_t src_qp;              - The remote QP number  
    int wc_flags;                 - Work Completion flags. Or of the following flags:  
                                IBV_WC_GRH – Indicator that the first 40 bytes of the receive buffer(s) contain a valid GRH  
                                IBV_WC_WITH_IMM – Indicator that the received message contains immediate data  
    uint16_t pkey_index;  
    uint16_t slid;                - For UD QP: the source LID  
    uint8_t sl;                   - For UD QP: the source Service Level  
    uint8_t dlid_path_bits;       - For UD QP: the destination LID path bits  
};
```

Polling for Work Completion: API

- Typical Work Completion status:
 - IBV_WC_SUCCESS – Operation completed successfully
 - IBV_WC_LOC_LEN_ERR – Local length error when processing SR or RR
 - IBV_WC_LOC_PROT_ERR – Local Protection error; S/G entries doesn't point to a valid MR
 - IBV_WC_WR_FLUSH_ERR – Work Request flush error; it was processed when the QP was in Error state
 - IBV_WC_RETRY_EXC_ERR – Retry exceeded; the remote QP didn't send any ACK/NACK, even after message retransmission
 - IBV_WC_RNR_RETRY_EXC_ERR – Receiver Not Ready; a message that requires a Receive Request was sent, but isn't any RR in the remote QP, even after message retransmission

“RC Pingpong” example – main loop

```
do {
    ne = ibv_poll_cq(ctx->cq, 2, wc);
} while (!use_event && ne < 1);

for (i = 0; i < ne; ++i) {
    switch ((int) wc[i].wr_id) {
    case PINGPONG_SEND_WRID:
        ++scnt;
        break;
    case PINGPONG_RECV_WRID:
        if (--routs <= 1) {
        }      routs += pp_post_recv(ctx, ctx->rx_depth - routs);
        ++rcnt;
        break;
    }
    ctx->pending &= ~(int) wc[i].wr_id;
    if (scnt < iters && !ctx->pending) {
        ctx->pending = PINGPONG_RECV_WRID |
                      PINGPONG_SEND_WRID;
    }
}
```

Questions?